📘 MODULE 7: SPL STATISTICS

7.1 Concept

stats **collapses many events into summary rows**. It is the most important command in SOC work: counting failed logins, finding the busiest IPs, and spotting outliers.

7.2 Syntax

spl

| stats <function>(<field>) [AS <name>] ... [by <field1>, <field2>]

7.3 Functions

Function	Meaning	Example
count	Number of events	stats count
dc(field)	**Distinct count** of unique values	stats dc(user)
values(field)	Unique values, sorted, de-duplicated	stats values(user)
list(field)	All values, in order, **including duplicates**	stats list(user)
avg(field)	Average (numeric)	stats avg(bytes)
sum(field)	Total	stats sum(bytes)
min(field) / max(field)	Smallest / largest	stats max(bytes)
earliest(field) / latest(field)	Value from the oldest / newest event	stats latest(status)
earliest(_time)	First time seen (epoch)	stats earliest(_time) AS first_seen

7.4 Worked queries

Query 1: total events

spl

index=_internal sourcetype=splunkd_access | stats count

What it does: counts all matching events and returns **one row**.
Common mistake: expecting events to remain. After stats, the raw events are gone.

Query 2: count by a field

spl

index=_internal sourcetype=splunkd_access

| stats count by clientip

| sort - count

What it does: one row per clientip with its event count, largest first.
Why useful: the SOC version is stats count by src_ip, which finds the noisiest attackers.
Interview explanation: "by splits the aggregation into groups, like GROUP BY in SQL."

Query 3: distinct count

spl

index=_internal sourcetype=splunkd_access

| stats dc(user) AS unique_users

What it does: how many *different* users appear. count counts events and dc counts unique values. This difference is a classic interview question.
SOC use: one IP failing logins for 50 different usernames is password spraying, and dc(user) by src_ip finds it.

Query 4: multiple functions and values

spl

index=_internal sourcetype=splunkd_access

| stats count AS requests, dc(uri_path) AS unique_paths, values(status) AS statuses, earliest(_time) AS first_seen, latest(_time) AS last_seen by clientip

| eval first_seen=strftime(first_seen,"%F %T"), last_seen=strftime(last_seen,"%F %T")

What it does: a profile per IP: volume, variety, status codes, first and last activity.
Common mistakes: avg() on non-numeric fields returns nothing. Time fields are epoch numbers, so format them with strftime.

values vs list

values is de-duplicated and sorted (compact). list keeps duplicates and order (limited to the first 100 values by default).

7.5 Performance and gotchas

- Always **filter first**, then stats.
- Fields not named in stats are discarded.

- stats by a field that is **missing** in an event drops that event from the output.
- High-cardinality by fields (e.g., one row per event ID) are expensive.

7.6 Hands-on exercises (write them yourself)

1. How many events are in _internal in the last hour?
2. Count splunkd events per log_level.
3. For splunkd_access, which user made the most requests?
4. For each clientip, how many unique uri_path values were requested?
5. For each host in index=*, show the first and last time it sent data.

Send me your SPL and I'll evaluate each answer.

7.7 Interview Q&A

- **count vs dc?** count counts events, dc counts distinct values.
- **stats vs top?** stats is general-purpose. top is a shortcut for frequency with percentages.
- **How do you find a source IP trying many usernames?** stats dc(user) by src_ip, then filter with where.
- **stats vs eventstats?** stats replaces events with results. eventstats adds the aggregate to each event (Module 14).

7.8 Revision

stats collapses events · by = GROUP BY · count=events, dc=unique · values dedups, list doesn't · time fields are epoch.